

GRNScope Optimization Brief Report

Large-matrix execution and memory-safety updates - June 2026

Objective

Reduce server freezes and memory blowups when users upload large expression matrices, especially datasets around 4,000+ genes and 10,000+ cells.

Main Changes

- Adaptive top-k edge output: algorithms now keep only the strongest regulators per target gene instead of writing all gene-by-gene edges.
- Pearson rewrite: blockwise correlation, float32 arrays, direct edge streaming, and no full correlation matrix written to disk.
- Shared BEELINE runner capping: large edge and matrix outputs are compacted across supported algorithms.
- Streaming confidence aggregation: each confidence run is read, aggregated, and released instead of loading all run outputs at once.
- Adaptive confidence runs: smaller datasets keep more bootstraps; larger datasets use fewer runs automatically.
- Arboreto controls: GENIE3 and GRNBoost2 now use capped outputs, controlled Dask workers, and safer float32 parsing.

Default Scaling Rules

Dataset size	Edges kept per target	Confidence runs
<= 500 genes	100	30
501-2000 genes	50	20
2001-3000 genes	50	10
3001-8000 genes	25	10
> 8000 genes	10; stronger gene filtering recommended	10

Expected Impact

- Memory demand changes from approximately $O(\text{genes squared})$ edge storage to $O(\text{genes} \times k)$ for final outputs.
- A 4,000-gene run drops from about 16 million possible directed edges to roughly 100,000 edges when $k = 25$.
- Confidence mode is less likely to overload the server because run outputs are compact and aggregated incrementally.
- Server behavior is more predictable through worker and Dask concurrency limits.

Tradeoff And Deployment Note

The system now prioritizes high-confidence, top-ranked regulators and does not preserve the full all-vs-all edge table by default. This is appropriate for interactive GRN inspection and prevents oversized outputs. Backend and runner updates are already committed; Docker images only need rebuilding for algorithm code that is baked into images rather than mounted at runtime.